

# Sharpness Lean Formalization Report

## Natural-language theorem inventory and dependency graph

Generated from local Lean sources on 2026-06-25 01:31.

## Proof Completion Check

- `lake env lean Sharpness/Main.lean`: passed.
- `lake build`: passed. The emitted messages were style/header linter warnings, not proof errors.
- A source scan over all Lean files found no `sorry`, `axiom`, `constant`, `unsafe`, `admit`, or global `set_option autoImplicit true`.
- `#print axioms Sharpness.sharpness_zd` returned exactly `{propext, Classical.choice, Quot.sound}`. In particular, no `sorryAx` appears.

The report covers 19 Sharpness modules, 85 definitions/structures/abbreviations, and 283 theorems/lemmas. It includes 166 private supporting declarations.

## Final Theorem

The Lean theorem `Sharpness.sharpness_zd` proves the following sharpness statement for nearest-neighbor Bernoulli bond percolation on  $Z^d$ .

For every  $d \geq 2$ :

$$\forall p, \quad 0 \leq p < p_c(d) \implies \exists c > 0, \forall n \in \mathbb{N}, \mathbb{P}_p(0 \leftrightarrow \partial \Lambda_n) \leq e^{-cn},$$

and

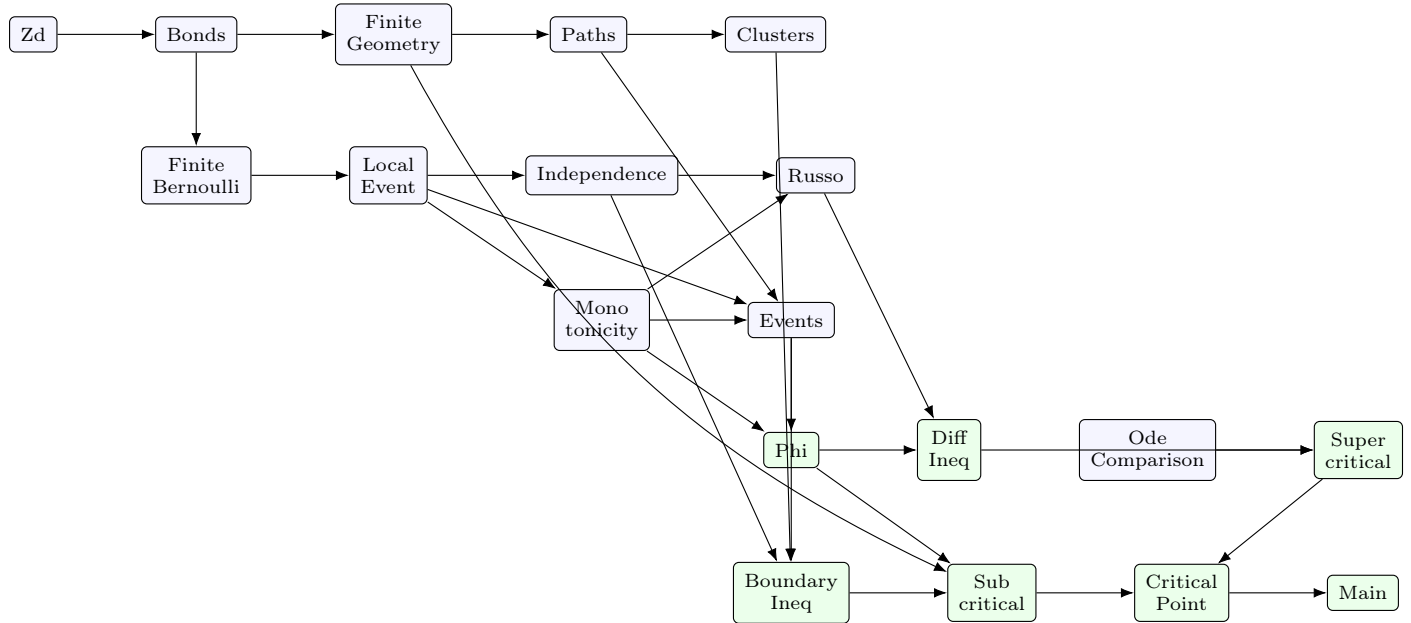
$$\forall p, \quad p_c(d) < p < 1 \implies \theta_d(p) \geq \frac{p - p_c(d)}{p(1 - p_c(d))}.$$

In Lean,  $\mathbb{P}_p(0 \leftrightarrow \partial \Lambda_n)$  is `boxExitProb d p n`,  $\theta_d(p)$  is `theta d p`, and  $p_c(d)$  is `pCrit d`.

## Core Theorems in Mathematical Language

| Name                                                                 | Natural-language mathematical statement                                                                                                                                                                                                                           |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>russo_closed_pivotal</code>                                    | Russo's formula for finite local increasing events: the derivative of the event probability equals the closed-pivotal probability sum divided by $1-p$ .                                                                                                          |
| <code>differential_inequality</code>                                 | For a finite volume $\Lambda$ containing the origin, the derivative of the exit probability is bounded below by $\phi_{\min}(p, \Lambda) / (p(1-p))$ times one minus the exit probability.                                                                        |
| <code>ode_log_comparison</code>                                      | A real-valued function satisfying the differential lower bound $f'(t) \geq (1 - f(t)) / (t(1 - t))$ obeys the corresponding logarithmic comparison inequality between $q$ and $p$ .                                                                               |
| <code>supercritical_lower_bound_</code><br><code>above_pTilde</code> | Above the finite-set critical point $p_{\text{Tilde}}$ and below 1, $\theta$ is bounded below by the explicit mean-field expression $(p - p_{\text{Tilde}}) / (p(1 - p_{\text{Tilde}}))$ .                                                                        |
| <code>boundary_inequality</code>                                     | The probability of connecting from $u$ to $B$ inside $T$ is bounded by a sum over the oriented boundary of $S$ , with one factor for reaching the boundary point in $S$ , one factor $p$ for the boundary bond, and one factor for connecting onward inside $T$ . |
| <code>exponential_decay_below_pTilde</code>                          | Below the finite-set critical point $p_{\text{Tilde}}$ , finite box exit probabilities decay exponentially.                                                                                                                                                       |
| <code>pTilde_eq_pCrit</code>                                         | In positive dimension, the finite-set critical point equals the $\theta$ -critical point.                                                                                                                                                                         |
| <code>sharpness_zd</code>                                            | For every dimension $d \geq 2$ , subcritical finite box exit probabilities decay exponentially below $p_c(d)$ , and above $p_c(d)$ the density $\theta_d(p)$ is bounded below by $(p - p_c(d)) / (p(1 - p_c(d)))$ .                                               |

# Module Dependency Graph



## Actual Import Edges

- `Sharpness.Zd` → `Sharpness.Bonds`
- `Sharpness.Bonds` → `Sharpness.FiniteGeometry`
- `Sharpness.FiniteGeometry` → `Sharpness.Paths`
- `Sharpness.FiniteGeometry` → `Sharpness.LocalEvent`
- `Sharpness.Paths` → `Sharpness.Clusters`
- `Sharpness.Bonds` → `Sharpness.FiniteBernoulli`
- `Sharpness.FiniteBernoulli` → `Sharpness.LocalEvent`
- `Sharpness.LocalEvent` → `Sharpness.Independence`
- `Sharpness.LocalEvent` → `Sharpness.Monotonicity`
- `Sharpness.Independence` → `Sharpness.Russo`
- `Sharpness.Monotonicity` → `Sharpness.Russo`
- `Sharpness.LocalEvent` → `Sharpness.Events`
- `Sharpness.Monotonicity` → `Sharpness.Events`
- `Sharpness.Paths` → `Sharpness.Events`
- `Sharpness.Events` → `Sharpness.Phi`
- `Sharpness.Monotonicity` → `Sharpness.Phi`
- `Sharpness.Phi` → `Sharpness.DiffIneq`
- `Sharpness.Russo` → `Sharpness.DiffIneq`
- `Sharpness.DiffIneq` → `Sharpness.Supercritical`
- `Sharpness.OdeComparison` → `Sharpness.Supercritical`
- `Sharpness.Clusters` → `Sharpness.BoundaryIneq`
- `Sharpness.Events` → `Sharpness.BoundaryIneq`
- `Sharpness.Independence` → `Sharpness.BoundaryIneq`
- `Sharpness.BoundaryIneq` → `Sharpness.Subcritical`
- `Sharpness.FiniteGeometry` → `Sharpness.Subcritical`
- `Sharpness.Phi` → `Sharpness.Subcritical`
- `Sharpness.Subcritical` → `Sharpness.CriticalPoint`
- `Sharpness.Supercritical` → `Sharpness.CriticalPoint`
- `Sharpness.CriticalPoint` → `Sharpness.Main`

## Per-File Inventory

`Sharpness/Zd.lean`

`Imports. Mathlib.`

## Definitions, structures, and abbreviations.

| Name                       | Natural-language mathematical statement                                                                        |
|----------------------------|----------------------------------------------------------------------------------------------------------------|
| Vertex                     | The vertex set of the lattice $\mathbb{Z}^d$ , represented as functions from $\text{Fin } d$ to $\text{Int}$ . |
| $\text{l1}$                | The $\text{l1}$ norm on $\mathbb{Z}^d$ , computed as the sum of absolute coordinate values.                    |
| Adj                        | Nearest-neighbor adjacency: two vertices are adjacent exactly when their $\text{l1}$ distance is one.          |
| translateVertex            | Translation of a lattice vertex by another lattice vector.                                                     |
| coordStep                  | A positive or negative unit coordinate step.                                                                   |
| coordRange [noncomputable] | The finite integer interval $[-n, n]$ used to build finite boxes.                                              |
| cube [noncomputable]       | The finite coordinate cube $[-n, n]^d$ .                                                                       |
| ball [noncomputable]       | The finite $\text{l1}$ ball $\text{Ball}_n = \{x \in \mathbb{Z}^d : \ x\ _1 \leq n\}$ .                        |
| neighbors [noncomputable]  | The finite set of nearest-neighbor candidates of a vertex.                                                     |

## Theorems and lemmas.

| Name                          | Natural-language mathematical statement                                                                                                                                    |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| translateVertex_apply         | Theorem for translate Vertex apply: $\text{translateVertex } a \ x \ i = x \ i + a \ i$ .                                                                                  |
| $\text{l1\_zero}$             | Theorem for $\text{l1}$ zero: $\text{l1 } (0 : \text{Vertex } d) = 0$ .                                                                                                    |
| $\text{l1\_neg}$              | Theorem for $\text{l1}$ neg: $\text{l1 } (-x) = \text{l1 } x$ .                                                                                                            |
| $\text{l1\_add\_le}$          | Theorem proving an inequality for $\text{l1}$ add le: $\text{l1 } (x + y)$ is at most $\text{l1 } x + \text{l1 } y$ .                                                      |
| $\text{l1\_sub\_comm}$        | Theorem for $\text{l1}$ sub comm: $\text{l1 } (x - y) = \text{l1 } (y - x)$ .                                                                                              |
| adj_symm                      | Theorem proving symmetry for adj symm: $\text{Adj } y \ x$ .                                                                                                               |
| ne_of_adj                     | Theorem for ne of adj: $x$ is not equal to $y$ .                                                                                                                           |
| mem_cube_iff                  | Theorem characterizing membership for mem cube iff: $x$ belongs to cube $d \ n$ if and only if for every $i : \text{Fin } d$ , $x \ i$ belongs to $\text{coordRange } n$ . |
| coord_natAbs_le_l1            | Theorem proving an inequality for coord nat Abs le $\text{l1}$ : $\text{Int.natAbs } (x \ i)$ is at most $\text{l1 } x$ .                                                  |
| coord_mem_coordRange_of_l1_le | Theorem proving an inequality for coord mem coord Range of $\text{l1}$ le: $x \ i$ belongs to $\text{coordRange } n$ .                                                     |
| mem_ball_iff                  | Characterize membership in the finite $\text{l}^1$ ball constructed from the bounded cube.                                                                                 |
| zero_mem_ball                 | The origin belongs to every finite $\text{l}^1$ ball.                                                                                                                      |
| adj_translate_iff             | Nearest-neighbor adjacency is invariant under translations.                                                                                                                |
| mem_neighbors_iff             | Nearest-neighbor candidates are exactly adjacent vertices.                                                                                                                 |
| translate_ball_exit           | The triangle estimate used to compare translated exit events.                                                                                                              |

Sharpness/Bonds.lean

Imports. Sharpness.Zd.

## Definitions, structures, and abbreviations.

| Name                             | Natural-language mathematical statement                                         |
|----------------------------------|---------------------------------------------------------------------------------|
| Bond                             | An unoriented nearest-neighbor bond, represented by its two-point endpoint set. |
| Config                           | A percolation configuration assigning each bond open or closed.                 |
| bondOfAdj                        | The unoriented bond associated to an adjacent ordered pair.                     |
| OrientedEdge                     | An oriented edge, represented as an ordered pair of vertices.                   |
| orientedBoundary [noncomputable] | The oriented boundary edges leaving a finite vertex set.                        |

| Name                                       | Natural-language mathematical statement                               |
|--------------------------------------------|-----------------------------------------------------------------------|
| <code>0EdgeBoundary</code>                 | A boundary-oriented edge bundled with its boundary membership proof.  |
| <code>0EdgeBoundary.bond</code>            | Forget the orientation of a boundary edge.                            |
| <code>internalBonds [noncomputable]</code> | The finite set of bonds with both endpoints inside a vertex set.      |
| <code>exitBonds [noncomputable]</code>     | The finite set of bonds crossing from a vertex set to its complement. |

#### Theorems and lemmas.

| Name                                     | Natural-language mathematical statement                                                                                                      |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>bondOfAdj_card_two</code>          | Adjacent vertices form a two-point finite carrier.                                                                                           |
| <code>bondOfAdj_adj_pair</code>          | Every distinct pair in the carrier $\{x,y\}$ is adjacent when $x$ and $y$ are adjacent.                                                      |
| <code>mem_orientedBoundary_iff</code>    | The finite neighbor construction exactly enumerates oriented nearest-neighbor boundary edges.                                                |
| <code>orientedBoundary_adj</code>        | Adjacency proof extracted from membership in the oriented boundary.                                                                          |
| <code>bondOfAdj_same</code>              | Theorem for <code>bond Of Adj same</code> : <code>bondOfAdj h = bondOfAdj h'</code> .                                                        |
| <code>bondOfAdj_symm</code>              | Theorem proving symmetry for <code>bond Of Adj symm</code> : <code>bondOfAdj (adj_symm hxy) = bondOfAdj hxy</code> .                         |
| <code>bondOfAdj_mem_internalBonds</code> | Theorem for <code>bond Of Adj mem internal Bonds</code> : <code>bondOfAdj hxy</code> belongs to <code>internalBonds S</code> .               |
| <code>bondOfAdj_mem_exitBonds</code>     | Theorem for <code>bond Of Adj mem exit Bonds</code> : <code>bondOfAdj (orientedBoundary_adj he)</code> belongs to <code>exitBonds S</code> . |
| <code>disjoint_internal_exit</code>      | Internal and exit bonds of the same finite set are disjoint.                                                                                 |

#### Sharpness/FiniteGeometry.lean

Imports. `Sharpness.Bonds`.

#### Definitions, structures, and abbreviations.

| Name                                          | Natural-language mathematical statement                           |
|-----------------------------------------------|-------------------------------------------------------------------|
| <code>boxInternalBonds [noncomputable]</code> | Internal bonds of the box <code>Lambda_n</code> .                 |
| <code>boxExitBonds [noncomputable]</code>     | Exit bonds crossing from <code>Lambda_n</code> to its complement. |

#### Theorems and lemmas.

| Name                                    | Natural-language mathematical statement                                                                                                                                                                                                                                                             |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>boundary_endpoint_mem_ball</code> | Boundary endpoints of a finite set contained in <code>Lambda_(L-1)</code> lie in <code>Lambda_L</code> . The positivity hypothesis is needed because <code>Nat</code> subtraction saturates: at $L = 0$ , $L - 1 = 0$ , while a boundary endpoint of $\{0\}$ need not be in <code>Lambda_0</code> . |

#### Sharpness/Paths.lean

Imports. `Sharpness.FiniteGeometry`.

#### Definitions, structures, and abbreviations.

| Name                    | Natural-language mathematical statement                                       |
|-------------------------|-------------------------------------------------------------------------------|
| <code>IsPath</code>     | The predicate that a list of vertices is a nearest-neighbor path.             |
| <code>PathFromTo</code> | The predicate that a path starts at one vertex and ends at another.           |
| <code>PathIn</code>     | The predicate that every vertex of a path lies in a finite set.               |
| <code>OpenPath</code>   | The predicate that every edge traversed by a path is open in a configuration. |
| <code>ConnIn</code>     | Connectivity inside a finite set by an open path.                             |

## Theorems and lemmas.

| Name                                         | Natural-language mathematical statement                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>first_exit_step_of_path</code>         | A path that starts in S and ends outside S has a crossing step.                                                                                                                                                                                                                                                                                                                                                |
| <code>last_exit_step_of_path</code>          | A path crossing from S to its complement has a boundary step; this alias is used for last-exit arguments before the later proof refines the chosen crossing.                                                                                                                                                                                                                                                   |
| <code>isPath_reverse</code>                  | Theorem for is Path reverse: <code>IsPath gamma.reverse</code> .                                                                                                                                                                                                                                                                                                                                               |
| <code>pathFromTo_reverse</code>              | Theorem for path From To reverse: <code>PathFromTo gamma.reverse y x</code> .                                                                                                                                                                                                                                                                                                                                  |
| <code>pathIn_reverse</code>                  | Theorem for path In reverse: <code>PathIn S gamma.reverse</code> .                                                                                                                                                                                                                                                                                                                                             |
| <code>openPath_reverse</code>                | Theorem for open Path reverse: <code>OpenPath omega gamma.reverse</code> .                                                                                                                                                                                                                                                                                                                                     |
| <code>connIn_symm</code>                     | Reversing a path preserves open connectivity.                                                                                                                                                                                                                                                                                                                                                                  |
| <code>isPath_translate</code>                | Theorem for is Path translate: <code>IsPath (gamma.map (translateVertex a))</code> .                                                                                                                                                                                                                                                                                                                           |
| <code>pathFromTo_translate</code>            | Theorem for path From To translate: <code>PathFromTo (gamma.map (translateVertex a)) (translateVertex a x) (translateVertex a y)</code> .                                                                                                                                                                                                                                                                      |
| <code>pathIn_translate</code>                | Theorem for path In translate: <code>PathIn (S.image (translateVertex a)) (gamma.map (translateVertex a))</code> .                                                                                                                                                                                                                                                                                             |
| <code>openPath_translate_of_edge_open</code> | Theorem for open Path translate of edge open: $\{d : \text{Nat}\} \{\omega : \text{Config } d\} \{a : \text{Vertex } d\} \{\gamma : \text{List } (\text{Vertex } d)\} (\text{hopen\_translate} : \text{for every } \{u v : \text{Vertex } d\} (\text{huv} : \text{Adj } u v), \omega (\text{bondOfAdj } \text{huv}) = \text{true} \text{ implies } \omega (\text{bondOfAdj } ((\text{adj\_translate\_iff } a.$ |
| <code>connIn_translate</code>                | Translated paths preserve open connectivity when translated open edges remain open.                                                                                                                                                                                                                                                                                                                            |

## Sharpness/Clusters.lean

Imports. `Sharpness.LocalEvent`, `Sharpness.Paths`.

## Definitions, structures, and abbreviations.

| Name                                              | Natural-language mathematical statement                                           |
|---------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>clusterIn [noncomputable]</code>            | The open cluster of a vertex inside a finite set.                                 |
| <code>ClusterEqPred</code>                        | The event that a finite set is exactly the cluster of a vertex in an ambient set. |
| <code>clusterIncidentBonds [noncomputable]</code> | Bonds incident to a finite cluster inside an ambient set.                         |

## Theorems and lemmas.

| Name                                                     | Natural-language mathematical statement                                                                                   |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>bondOfAdj_mem_</code>                              | Private supporting lemma for bond Of Adj mem cluster Incident Bonds:                                                      |
| <code>clusterIncidentBonds [private]</code>              | <code>bondOfAdj hxy</code> belongs to <code>clusterIncidentBonds S C</code> .                                             |
| <code>connIn_single [private]</code>                     | Private supporting lemma for conn In single: <code>ConnIn omega S x x</code> .                                            |
| <code>connIn_append_open_adj_same [private]</code>       | Private supporting lemma for conn In append open adj same: <code>ConnIn omega S x z</code> .                              |
| <code>mem_cluster_of_conn [private]</code>               | Private supporting lemma for mem cluster of conn: <code>x</code> belongs to <code>C</code> .                              |
| <code>conn_of_mem_cluster [private]</code>               | Private supporting lemma for conn of mem cluster: <code>ConnIn omega S u x</code> .                                       |
| <code>target_mem_of_connIn [private]</code>              | Private supporting lemma for target mem of conn In: <code>x</code> belongs to <code>S</code> .                            |
| <code>mem_clusterIn_of_conn [private]</code>             | Private supporting lemma for mem cluster In of conn: <code>x</code> belongs to <code>clusterIn omega S u</code> .         |
| <code>openPath_transfer_from_cluster_eq [private]</code> | Private supporting lemma proving an equality for open Path transfer from cluster eq: <code>OpenPath omega' gamma</code> . |
| <code>openPath_transfer_to_cluster_eq [private]</code>   | Private supporting lemma proving an equality for open Path transfer to cluster eq: <code>OpenPath omega gamma</code> .    |
| <code>connIn_transfer_of_cluster_eq [private]</code>     | Private supporting lemma proving an equality for conn In transfer of cluster eq: <code>ConnIn omega' S u x</code> .       |

| Name                                  | Natural-language mathematical statement                                                                                                                                |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| connIn_other_subset_cluster [private] | Private supporting lemma for conn In other subset cluster: x belongs to C.                                                                                             |
| cluster_eq_dependsOn_incident         | Theorem proving that the event is local with the stated finite support for cluster eq depends On incident: DependsOn (clusterIncidentBonds S C) (ClusterEqPred S C u). |

Sharpness/FiniteBernoulli.lean

Imports. Sharpness.Bonds.

Definitions, structures, and abbreviations.

| Name                       | Natural-language mathematical statement                                             |
|----------------------------|-------------------------------------------------------------------------------------|
| bernWeight [noncomputable] | The Bernoulli product weight of a finite Boolean assignment.                        |
| bernProb [noncomputable]   | The finite Bernoulli product probability of a Boolean event.                        |
| weight [noncomputable]     | The Bernoulli product weight on a finite set of bonds.                              |
| ProbOn [noncomputable]     | The finite Bernoulli probability of a predicate on assignments over a bond support. |

Theorems and lemmas.

| Name                 | Natural-language mathematical statement                                                                                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bernWeight_nonneg    | Theorem proving nonnegativity for bern Weight nonneg: 0 is at most bernWeight p sigma.                                                                                                                 |
| bernWeight_total     | Theorem for bern Weight total: (Finset.univ.sum fun sigma : alpha implies Bool => bernWeight p sigma) = 1.                                                                                             |
| bernProb_nonneg      | Theorem proving nonnegativity for bern Prob nonneg: 0 is at most bernProb p A.                                                                                                                         |
| bernProb_le_one      | Theorem proving an upper bound by one for bern Prob le one: bernProb p A is at most 1.                                                                                                                 |
| bernProb_univ        | Theorem for bern Prob univ: bernProb p (fun _ : alpha implies Bool => True) = 1.                                                                                                                       |
| bernProb_compl       | Theorem for bern Prob compl: bernProb p (fun sigma => not A sigma) = 1 - bernProb p A.                                                                                                                 |
| bernProb_mono        | Theorem proving a monotonicity property for bern Prob mono: bernProb p A is at most bernProb p B.                                                                                                      |
| bernProb_union_bound | Theorem proving a finite union bound for bern Prob union bound: bernProb p (fun sigma => A sigma or B sigma) is at most bernProb p A + bernProb p B.                                                   |
| bernWeight_reindex   | Theorem for bern Weight reindex: bernWeight p (fun a : alpha => sigma (e a)) = bernWeight p sigma.                                                                                                     |
| bernProb_reindex     | Theorem for bern Prob reindex: bernProb p (fun sigma : alpha implies Bool => A (fun b : beta => sigma (e.symm b))) = bernProb p A.                                                                     |
| bernWeight_sum       | Theorem for bern Weight sum: bernWeight p sigma = bernWeight p (fun a : alpha => sigma (Sum.inl a)) * bernWeight p (fun b : beta => sigma (Sum.inr b)).                                                |
| bernProb_sum_left    | Theorem for bern Prob sum left: bernProb p (fun sigma : alpha + beta implies Bool => A (fun a : alpha => sigma (Sum.inl a))) = bernProb p A.                                                           |
| bernProb_sum_inter   | Theorem for bern Prob sum inter: bernProb p (fun sigma : alpha + beta implies Bool => A (fun a : alpha => sigma (Sum.inl a)) and B (fun b : beta => sigma (Sum.inr b))) = bernProb p A * bernProb p B. |
| probOn_nonneg        | Theorem proving nonnegativity for prob On nonneg: 0 is at most ProbOn p F A.                                                                                                                           |
| probOn_le_one        | Theorem proving an upper bound by one for prob On le one: ProbOn p F A is at most 1.                                                                                                                   |
| probOn_univ          | Theorem for prob On univ: ProbOn p F (fun _ => True) = 1.                                                                                                                                              |
| probOn_compl         | Theorem for prob On compl: ProbOn p F (fun sigma => not A sigma) = 1 - ProbOn p F A.                                                                                                                   |

| Name               | Natural-language mathematical statement                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| probOn_mono        | Theorem proving a monotonicity property for prob On mono: ProbOn p F A is at most ProbOn p F B.                                                    |
| probOn_union_bound | Theorem proving a finite union bound for prob On union bound: ProbOn p F (fun sigma => A sigma or B sigma) is at most ProbOn p F A + ProbOn p F B. |

Sharpness/LocalEvent.lean

Imports. Sharpness.FiniteBernoulli.

Definitions, structures, and abbreviations.

| Name                                           | Natural-language mathematical statement                                                   |
|------------------------------------------------|-------------------------------------------------------------------------------------------|
| DependsOn                                      | A locality predicate saying an event depends only on a finite bond support.               |
| LocalEvent                                     | A percolation event together with an explicit finite support and locality proof.          |
| extendConfig [noncomputable]                   | Extends a finite support assignment to a global configuration by closing all other bonds. |
| Prob [noncomputable]                           | The probability of a local event computed on its finite support.                          |
| supportExtensionEquiv [private, noncomputable] | Private auxiliary definition named support Extension Equiv.                               |
| LocalEvent.compl                               | Complement of a local event.                                                              |
| LocalEvent.inter                               | Intersection of two local events.                                                         |
| LocalEvent.union                               | Union of two local events.                                                                |

Theorems and lemmas.

| Name                                      | Natural-language mathematical statement                                                                                                                |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| supportExtensionEquiv_symm_left [private] | Private supporting lemma proving symmetry for support Extension Equiv symm left: (supportExtensionEquiv F G hsub).symm <e, hsub he> = Sum.inl <e, he>. |
| prob_support_mono                         | Theorem proving a monotonicity property for prob support mono: Prob p E = ProbOn p G (fun sigma => E.pred (extendConfig G sigma)).                     |

Sharpness/Independence.lean

Imports. Sharpness.LocalEvent.

Definitions, structures, and abbreviations. *None.*

Theorems and lemmas.

| Name                                         | Natural-language mathematical statement                                                                                                      |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| prob_inter_eq_mul_of_disjoint                | Theorem proving an equality for prob inter eq mul of disjoint: Prob p (E.inter F) = Prob p E * Prob p F.                                     |
| prob_inter_three_eq_mul_of_pairwise_disjoint | Theorem proving an equality for prob inter three eq mul of pairwise disjoint: Prob p ((E.inter F).inter G) = Prob p E * Prob p F * Prob p G. |

Sharpness/Monotonicity.lean

Imports. Sharpness.LocalEvent.

Definitions, structures, and abbreviations.

| Name     | Natural-language mathematical statement                                      |
|----------|------------------------------------------------------------------------------|
| ConfigLE | The pointwise partial order on configurations: open bonds can only be added. |

| Name                | Natural-language mathematical statement                                |
|---------------------|------------------------------------------------------------------------|
| Increasing          | A local event is increasing if opening more bonds preserves the event. |
| BoolAssignmentLE    | The pointwise Boolean order on finite assignments.                     |
| IncreasingBoolEvent | An increasing event on a finite Boolean cube.                          |

#### Theorems and lemmas.

| Name                                         | Natural-language mathematical statement                                                                                                                                                                                                                                                                     |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bernWeight_fin_cons [private]                | Private supporting lemma for bern Weight fin cons: $\text{bernWeight } p \text{ (Fin.cons } b \text{ tau)} = (\text{if } b \text{ then } p \text{ else } 1 - p) * \text{bernWeight } p \text{ tau}.$                                                                                                        |
| bernProb_fin_succ [private]                  | Private supporting lemma for bern Prob fin succ: $\text{bernProb } p \text{ A} = (1 - p) * \text{bernProb } p \text{ (fun tau : Fin n implies Bool } \Rightarrow \text{A (Fin.cons false tau))} + p * \text{bernProb } p \text{ (fun tau : Fin n implies Bool } \Rightarrow \text{A (Fin.cons true tau))}.$ |
| IncreasingBoolEvent_fin_tail_false [private] | Private supporting lemma for Increasing Bool Event fin tail false: $\text{IncreasingBoolEvent (fun tau : Fin n implies Bool } \Rightarrow \text{A (Fin.cons false tau))}.$                                                                                                                                  |
| IncreasingBoolEvent_fin_tail_true [private]  | Private supporting lemma for Increasing Bool Event fin tail true: $\text{IncreasingBoolEvent (fun tau : Fin n implies Bool } \Rightarrow \text{A (Fin.cons true tau))}.$                                                                                                                                    |
| fin_false_le_true [private]                  | Private supporting lemma proving an inequality for fin false le true: $\text{BoolAssignmentLE (Fin.cons false tau) (Fin.cons true tau)}.$                                                                                                                                                                   |
| bernProb_fin_mono_parameter [private]        | Private supporting lemma proving a monotonicity property for bern Prob fin mono parameter: $\text{bernProb } p \text{ A}$ is at most $\text{bernProb } q \text{ A}.$                                                                                                                                        |
| bernProb_mono_parameter                      | Theorem proving a monotonicity property for bern Prob mono parameter: $\text{bernProb } p \text{ A}$ is at most $\text{bernProb } q \text{ A}.$                                                                                                                                                             |
| extendConfig_mono                            | Theorem proving a monotonicity property for extend Config mono: $\text{ConfigLE (extendConfig F sigma) (extendConfig F tau)}.$                                                                                                                                                                              |
| prob_mono                                    | Theorem proving a monotonicity property for prob mono: $\text{Prob } p \text{ E}$ is at most $\text{Prob } q \text{ E}.$                                                                                                                                                                                    |

#### Sharpness/Russo.lean

Imports. Sharpness.Independence, Sharpness.Monotonicity.

#### Definitions, structures, and abbreviations.

| Name                                 | Natural-language mathematical statement                                              |
|--------------------------------------|--------------------------------------------------------------------------------------|
| indic [noncomputable]                | A real-valued indicator expression.                                                  |
| forceBool                            | Forces one Boolean coordinate to a chosen value.                                     |
| ClosedPivotalBool                    | Closed-pivotality of a coordinate for a finite increasing Boolean event.             |
| configAt                             | Reconstructs a Boolean assignment from one coordinate and the remaining coordinates. |
| deleteCoord                          | Deletes one coordinate from a Boolean assignment.                                    |
| boolFactor                           | The one-coordinate factor in the Bernoulli product weight.                           |
| bernWeightScoreCoord [noncomputable] | The coordinate score term used to differentiate Bernoulli product weights.           |
| splitAt                              | An equivalence splitting a Boolean cube at one coordinate.                           |
| forceOpen                            | Forces a bond to be open in a percolation configuration.                             |
| ClosedPivotal                        | The event that a closed bond becomes pivotal when forced open.                       |
| closedPivotalEvent                   | The closed-pivotal event packaged as a local event.                                  |

#### Theorems and lemmas.

| Name          | Natural-language mathematical statement                                |
|---------------|------------------------------------------------------------------------|
| configAt_self | Theorem for config At self: $\text{configAt } e \text{ b tau } e = b.$ |



| Name                                 | Natural-language mathematical statement                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| configAt_ne                          | Theorem for config At ne: $\text{configAt } e \text{ b tau } f = \text{tau } \langle f, \text{hfe} \rangle$ .                                                                                                                                                                                                                                                                                                                                                                                                  |
| deleteCoord_configAt                 | Theorem for delete Coord config At: $\text{deleteCoord } e (\text{configAt } e \text{ b tau}) = \text{tau}$ .                                                                                                                                                                                                                                                                                                                                                                                                  |
| forceBool_configAt_false             | Theorem for force Bool config At false: $\text{forceBool } e \text{ true } (\text{configAt } e \text{ false tau}) = \text{configAt } e \text{ true tau}$ .                                                                                                                                                                                                                                                                                                                                                     |
| erase_prod_boolFactor_eq_deleteCoord | Theorem proving an equality for erase prod bool Factor eq delete Coord:<br>$((\text{Finset.univ.erase } e).\text{prod fun } f : \alpha \Rightarrow \text{boolFactor sigma } f \text{ p}) = \text{bernWeight } p (\text{deleteCoord } e \text{ sigma})$ .                                                                                                                                                                                                                                                       |
| bernWeight_hasDerivAt_score          | Theorem for bern Weight has Deriv At score: $\text{HasDerivAt } (\text{fun } q : \text{Real} \Rightarrow \text{bernWeight } q \text{ sigma}) (\text{Finset.univ.sum fun } e : \alpha \Rightarrow \text{bernWeightScoreCoord } p \text{ sigma } e) \text{ p}$ .                                                                                                                                                                                                                                                 |
| bernWeight_configAt_false            | Theorem for bern Weight config At false: $\text{bernWeight } p (\text{configAt } e \text{ false tau}) = (1 - p) * \text{bernWeight } p \text{ tau}$ .                                                                                                                                                                                                                                                                                                                                                          |
| bernWeight_configAt_true             | Theorem for bern Weight config At true: $\text{bernWeight } p (\text{configAt } e \text{ true tau}) = p * \text{bernWeight } p \text{ tau}$ .                                                                                                                                                                                                                                                                                                                                                                  |
| scoreCoord_configAt_false            | Theorem for score Coord config At false: $\text{bernWeightScoreCoord } p (\text{configAt } e \text{ false tau}) \text{ e} = -\text{bernWeight } p \text{ tau}$ .                                                                                                                                                                                                                                                                                                                                               |
| scoreCoord_configAt_true             | Theorem for score Coord config At true: $\text{bernWeightScoreCoord } p (\text{configAt } e \text{ true tau}) \text{ e} = \text{bernWeight } p \text{ tau}$ .                                                                                                                                                                                                                                                                                                                                                  |
| score_pair_eq_pivotal                | Theorem proving an equality for score pair eq pivotal: $\text{indic } (A (\text{configAt } e \text{ false tau})) (\text{bernWeightScoreCoord } p (\text{configAt } e \text{ false tau}) \text{ e}) + \text{indic } (A (\text{configAt } e \text{ true tau})) (\text{bernWeightScoreCoord } p (\text{configAt } e \text{ true tau}) \text{ e}) = \text{indic } (\text{ClosedPivotalBool } A \text{ e } (\text{configAt } e \text{ false tau})) (\text{bernWeight } p \text{ tau})$ .                            |
| closedPivotal_configAt_true_false    | Theorem for closed Pivotal config At true false: $\text{not ClosedPivotalBool } A \text{ e } (\text{configAt } e \text{ true tau})$ .                                                                                                                                                                                                                                                                                                                                                                          |
| closedPivotal_prob_pair              | Theorem for closed Pivotal prob pair: $\text{indic } (\text{ClosedPivotalBool } A \text{ e } (\text{configAt } e \text{ false tau})) (\text{bernWeight } p (\text{configAt } e \text{ false tau})) + \text{indic } (\text{ClosedPivotalBool } A \text{ e } (\text{configAt } e \text{ true tau})) (\text{bernWeight } p (\text{configAt } e \text{ true tau})) = (1 - p) * \text{indic } (\text{ClosedPivotalBool } A \text{ e } (\text{configAt } e \text{ false tau})) (\text{bernWeight } p \text{ tau})$ . |
| scoreCoord_sum_eq_closedPivotal      | Theorem proving an equality for score Coord sum eq closed Pivotal:<br>$(\text{Finset.univ.sum fun sigma : alpha implies Bool} \Rightarrow \text{indic } (A \text{ sigma}) (\text{bernWeightScoreCoord } p \text{ sigma } e)) = (1 / (1 - p)) * \text{bernProb } p (\text{ClosedPivotalBool } A \text{ e})$ .                                                                                                                                                                                                   |
| indic_sum                            | Theorem for indic sum: $\text{indic } P (\text{Finset.univ.sum } f) = \text{Finset.univ.sum fun } e : \alpha \Rightarrow \text{indic } P (f \text{ e})$ .                                                                                                                                                                                                                                                                                                                                                      |
| bernProb_hasDerivAt_score            | Theorem for bern Prob has Deriv At score: $\text{HasDerivAt } (\text{fun } q : \text{Real} \Rightarrow \text{bernProb } q \text{ A}) (\text{Finset.univ.sum fun sigma : alpha implies Bool} \Rightarrow \text{indic } (A \text{ sigma}) (\text{Finset.univ.sum fun } e : \alpha \Rightarrow \text{bernWeightScoreCoord } p \text{ sigma } e)) \text{ p}$ .                                                                                                                                                     |
| score_total_eq_closedPivotal         | Theorem proving an equality for score total eq closed Pivotal: $(\text{Finset.univ.sum fun sigma : alpha implies Bool} \Rightarrow \text{indic } (A \text{ sigma}) (\text{Finset.univ.sum fun } e : \alpha \Rightarrow \text{bernWeightScoreCoord } p \text{ sigma } e)) = (1 / (1 - p)) * (\text{Finset.univ.sum fun } e : \alpha \Rightarrow \text{bernProb } p (\text{ClosedPivotalBool } A \text{ e}))$ .                                                                                                  |
| bernProb_russo_closed_pivotal        | Russo's formula on a finite Boolean cube in closed-pivotal form.                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| closedPivotal_dependsOn              | Theorem proving that the event is local with the stated finite support for closed Pivotal depends On: $\text{DependsOn } (\text{insert } e \text{ E.support}) (\text{ClosedPivotal } E \text{ e})$ .                                                                                                                                                                                                                                                                                                           |
| closedPivotal_restrict_iff           | Theorem giving an if-and-only-if characterization for closed Pivotal restrict iff:<br>$\text{ClosedPivotal } E \text{ e } (\text{extendConfig } E.\text{support } \text{sigma}) \text{ if and only if } \text{ClosedPivotalBool } (\text{fun sigma : E.support implies Bool} \Rightarrow E.\text{pred } (\text{extendConfig } E.\text{support } \text{sigma})) \langle e, \text{he} \rangle \text{ sigma}$ .                                                                                                   |
| closedPivotal_prob_restrict          | Theorem for closed Pivotal prob restrict: $\text{bernProb } p (\text{ClosedPivotalBool } (\text{fun sigma : E.support implies Bool} \Rightarrow E.\text{pred } (\text{extendConfig } E.\text{support } \text{sigma})) \text{ e}) = \text{Prob } p (\text{closedPivotalEvent } E \text{ e.1})$ .                                                                                                                                                                                                                |
| russo_closed_pivotal                 | Russo's formula for finite local increasing events: the derivative of the event probability equals the closed-pivotal probability sum divided by 1-p.                                                                                                                                                                                                                                                                                                                                                          |

## Sharpness/Events.lean

**Imports.** Sharpness.LocalEvent, Sharpness.Monotonicity, Sharpness.Paths.

### Definitions, structures, and abbreviations.

| Name                             | Natural-language mathematical statement                                               |
|----------------------------------|---------------------------------------------------------------------------------------|
| ExitPred                         | The finite exit event: the origin reaches an open boundary edge leaving a finite set. |
| connInEvent [noncomputable]      | Connectivity inside a finite set packaged as a local event.                           |
| exitEvent [noncomputable]        | The finite exit event packaged as a local event.                                      |
| exitProb [noncomputable]         | The finite-volume exit probability from a finite set.                                 |
| boxExitProb [noncomputable]      | The exit probability from the l1 ball $\Lambda_n$ .                                   |
| ConnToSetIn                      | Connectivity from a vertex to some vertex in a target set within an ambient set.      |
| connToSetInEvent [noncomputable] | The event of connecting to a target set inside an ambient finite set.                 |
| theta [noncomputable]            | The infimum over finite box exit probabilities, used as the decreasing-limit theta.   |

### Theorems and lemmas.

| Name                               | Natural-language mathematical statement                                                                                                                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| openPath_dependsOn_of_pathIn       | Theorem proving that the event is local with the stated finite support for open Path depends On of path In: $\text{OpenPath } \omega \text{ gamma}$ if and only if $\text{OpenPath } \omega \text{ gamma}'$ . |
| connIn_dependsOn                   | Finite-set connectivity depends only on bonds internal to the finite set.                                                                                                                                     |
| openPath_mono                      | Theorem proving a monotonicity property for open Path mono: $\text{OpenPath } \omega \text{ gamma}$ .                                                                                                         |
| connIn_mono                        | Theorem proving a monotonicity property for conn In mono: $\text{ConnIn } \omega \text{ gamma}' \text{ S x y}$ .                                                                                              |
| connInEvent_increasing             | Theorem proving monotonicity of the corresponding local event for conn In Event increasing: $\text{Increasing } (\text{connInEvent } \text{S x y})$ .                                                         |
| exitEvent_dependsOn                | The finite exit event is local on internal and boundary bonds of Lam.                                                                                                                                         |
| exitPred_mono                      | Theorem proving a monotonicity property for exit Pred mono: $\text{ExitPred } \text{Lam } \omega \text{ gamma}'$ .                                                                                            |
| exitEvent_increasing               | Theorem proving monotonicity of the corresponding local event for exit Event increasing: $\text{Increasing } (\text{exitEvent } \text{Lam } h0)$ .                                                            |
| connToSetIn_dependsOn              | Finite target connectivity is local on the ambient internal bonds.                                                                                                                                            |
| connToSetIn_mono                   | Theorem proving a monotonicity property for conn To Set In mono: $\text{ConnToSetIn } \omega \text{ gamma}' \text{ T u B}$ .                                                                                  |
| connToSetInEvent_increasing        | Theorem proving monotonicity of the corresponding local event for conn To Set In Event increasing: $\text{Increasing } (\text{connToSetInEvent } \text{T u B})$ .                                             |
| boxExitProb_nonneg                 | Theorem proving nonnegativity for box Exit Prob nonneg: 0 is at most $\text{boxExitProb } d \text{ p n}$ .                                                                                                    |
| theta_nonneg                       | Theorem proving nonnegativity for theta nonneg: 0 is at most $\text{theta } d \text{ p}$ .                                                                                                                    |
| theta_le_boxExitProb               | Theorem proving an inequality for theta le box Exit Prob: $\text{theta } d \text{ p}$ is at most $\text{boxExitProb } d \text{ p n}$ .                                                                        |
| theta_eq_zero_of_exponential_decay | Theorem proving an equality for theta eq zero of exponential decay: $\text{theta } d \text{ p} = 0$ .                                                                                                         |
| le_theta_of_le_boxExitProb         | Theorem proving an inequality for le theta of le box Exit Prob: $b$ is at most $\text{theta } d \text{ p}$ .                                                                                                  |

## Sharpness/Phi.lean

**Imports.** Sharpness.Events, Sharpness.Monotonicity.

### Definitions, structures, and abbreviations.

| Name                                               | Natural-language mathematical statement                                            |
|----------------------------------------------------|------------------------------------------------------------------------------------|
| <code>phi [noncomputable]</code>                   | The finite-set criterion quantity $\phi_p(S)$ .                                    |
| <code>finiteSubsetsWithZero [noncomputable]</code> | The finite subsets of $\Lambda$ that contain the origin.                           |
| <code>phiMinIn [noncomputable]</code>              | The minimum of $\phi_p(S)$ over finite $S$ subset $\Lambda$ containing the origin. |
| <code>pTildeSet [noncomputable]</code>             | The parameters satisfying the finite-set criterion.                                |
| <code>pTilde [noncomputable]</code>                | The finite-set critical point, defined as the supremum of <code>pTildeSet</code> . |

### Theorems and lemmas.

| Name                                        | Natural-language mathematical statement                                                               |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>zero_mem_pTildeSet</code>             | 0 belongs to the defining set of <code>pTilde</code> .                                                |
| <code>pTildeSet_nonempty</code>             | The defining set of <code>pTilde</code> is nonempty.                                                  |
| <code>pTildeSet_bddAbove</code>             | The defining set of <code>pTilde</code> is bounded above by 1.                                        |
| <code>zero_le_pTilde</code>                 | The finite-set critical point is nonnegative.                                                         |
| <code>pTilde_le_one</code>                  | The finite-set critical point is at most one.                                                         |
| <code>phi_mono</code>                       | For each finite $S$ , $\phi_p(S)$ is monotone in the Bernoulli parameter on $[0,1]$ .                 |
| <code>exists_phi_lt_one_of_lt_pTilde</code> | Below <code>pTilde</code> , some finite set satisfies the finite-set criterion.                       |
| <code>one_le_phi_of_pTilde_lt</code>        | Above <code>pTilde</code> , every finite set containing the origin violates the finite-set criterion. |

### Sharpness/DiffIneq.lean

Imports. `Sharpness.Phi`, `Sharpness.Russo`.

### Definitions, structures, and abbreviations.

| Name                                                   | Natural-language mathematical statement                                                      |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>ExitFromPred</code>                              | The event that a vertex connects to the boundary of a finite set.                            |
| <code>Shield [noncomputable]</code>                    | The random shield set obtained by deleting internal edges and looking at the origin cluster. |
| <code>closeInternalEdges [noncomputable]</code>        | The configuration obtained by closing all internal edges of a set.                           |
| <code>shieldSupport [noncomputable]</code>             | The finite bond support needed to determine a shield value.                                  |
| <code>shieldEvent [noncomputable]</code>               | The event that the random shield equals a specified finite set.                              |
| <code>finsetUnionEvent [private, noncomputable]</code> | Private auxiliary definition named <code>finset Union Event</code> .                         |

### Theorems and lemmas.

| Name                                                               | Natural-language mathematical statement                                                                                                                                |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>exitFrom_zero_iff_exitPred</code>                            | Exit from the origin is the finite exit predicate used by <code>exitEvent</code> .                                                                                     |
| <code>exitFrom_dependsOn</code>                                    | Exit from a fixed vertex is local on the finite exit support of $\Lambda$ .                                                                                            |
| <code>zero_mem_shield_iff_not_exit</code>                          | The origin belongs to the shield exactly when the finite exit event fails.                                                                                             |
| <code>bondOfAdj_mem_internalBonds_endpoints [private]</code>       | Private supporting lemma for <code>bond Of Adj mem internal Bonds</code> endpoints: $x$ belongs to $S$ and $y$ belongs to $S$ .                                        |
| <code>bondOfAdj_not_mem_internalBonds_of_not_both [private]</code> | Private supporting lemma for <code>bond Of Adj not mem internal Bonds</code> of not both: <code>bondOfAdj hxy</code> does not belong to <code>internalBonds S</code> . |
| <code>openPath_of_closeInternalEdges [private]</code>              | Private supporting lemma for <code>open Path of close Internal Edges</code> : <code>OpenPath omega gamma</code> .                                                      |

| Name                                                 | Natural-language mathematical statement                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| connIn_of_closeInternalEdges [private]               | Private supporting lemma for conn In of close Internal Edges: ConnIn omega Lam x y.                                                                                                                                                                                                                                                                                                                                         |
| exitFrom_of_closeInternalEdges [private]             | Private supporting lemma for exit From of close Internal Edges: ExitFromPred Lam x omega.                                                                                                                                                                                                                                                                                                                                   |
| connIn_single [private]                              | Private supporting lemma for conn In single: ConnIn omega S x x.                                                                                                                                                                                                                                                                                                                                                            |
| openPath_closeInternalEdges_of_no_internal [private] | Private supporting lemma for open Path close Internal Edges of no internal: OpenPath (closeInternalEdges S omega) gamma.                                                                                                                                                                                                                                                                                                    |
| openPath_closeInternalEdges_of_unique_S [private]    | Private supporting lemma for open Path close Internal Edges of unique S: OpenPath (closeInternalEdges S omega) gamma.                                                                                                                                                                                                                                                                                                       |
| exists_closed_conn_from_last_S [private]             | Private supporting lemma proving existence of the stated object for exists closed conn from last S: for every {gamma : List (Vertex d)} {x y : Vertex d}, PathFromTo gamma x y implies PathIn Lam gamma implies OpenPath omega gamma implies (there exists z, z belongs to gamma and z belongs to S) implies y does not belong to S implies there exists u, u belongs to S and ConnIn (closeInternalEdges S omega) Lam u y. |
| exists_exitFrom_close_of_exitFrom_from_S [private]   | Private supporting lemma proving existence of the stated object for exists exit From close of exit From from S: there exists u, u belongs to S and ExitFromPred Lam u (closeInternalEdges S omega).                                                                                                                                                                                                                         |
| exitFrom_close_of_exitFrom_of_no_S_exit [private]    | Private supporting lemma for exit From close of exit From of no S exit: ExitFromPred Lam x (closeInternalEdges S omega).                                                                                                                                                                                                                                                                                                    |
| disjoint_internalBonds_shieldSupport                 | Internal S-bonds are disjoint from the support used by the shield event.                                                                                                                                                                                                                                                                                                                                                    |
| shield_dependsOn_ambient                             | The shield value is local on the ambient finite exit support.                                                                                                                                                                                                                                                                                                                                                               |
| shield_eq_iff_deleted_internal                       | Deleted-internal-edge characterization of {Shield = S}. After closing all internal S-edges, shield equality is equivalent to no S-vertex exiting and every Lam \ S vertex exiting.                                                                                                                                                                                                                                          |
| shield_dependsOn_noninternal                         | Support separation for the shield event {Shield = S}. This combines shield_eq_iff_deleted_internal with locality of finite exit events in the deleted configuration to show that internal S-edges are irrelevant.                                                                                                                                                                                                           |
| connIn_subset [private]                              | Private supporting lemma for conn In subset: ConnIn omega T x y.                                                                                                                                                                                                                                                                                                                                                            |
| connIn_append_open_adj_same [private]                | Private supporting lemma for conn In append open adj same: ConnIn omega S x z.                                                                                                                                                                                                                                                                                                                                              |
| connIn_cons_open_adj [private]                       | Private supporting lemma for conn In cons open adj: ConnIn omega S x z.                                                                                                                                                                                                                                                                                                                                                     |
| connIn_trans [private]                               | Private supporting lemma for conn In trans: ConnIn omega S x z.                                                                                                                                                                                                                                                                                                                                                             |
| connIn_forceOpen_of_connIn [private]                 | Private supporting lemma for conn In force Open of conn In: ConnIn (forceOpen b omega) S x y.                                                                                                                                                                                                                                                                                                                               |
| connIn_of_forceOpen_of_not_internal [private]        | Private supporting lemma for conn In of force Open of not internal: ConnIn omega S x y.                                                                                                                                                                                                                                                                                                                                     |
| first_exit_conn_of_path [private]                    | Private supporting lemma for first exit conn of path: for every {gamma : List (Vertex d)} {x y : Vertex d}, PathFromTo gamma x y implies OpenPath omega gamma implies x belongs to S implies y does not belong to S implies there exists a c : Vertex d, there exists hac : Adj a c, ConnIn omega S x a and a belongs to S and c does not belong to S and omega (bondOfAdj hac) = true.                                     |
| exitFrom_mono [private]                              | Private supporting lemma proving a monotonicity property for exit From mono: ExitFromPred Lam x omega'.                                                                                                                                                                                                                                                                                                                     |
| exitPred_of_connIn_boundary_open [private]           | Private supporting lemma for exit Pred of conn In boundary open: ExitPred Lam omega.                                                                                                                                                                                                                                                                                                                                        |
| exitPred_of_connIn_shield_boundary_open [private]    | Private supporting lemma for exit Pred of conn In shield boundary open: ExitPred Lam omega.                                                                                                                                                                                                                                                                                                                                 |
| boundary_left_eq_of_bond_eq [private]                | Private supporting lemma proving an equality for boundary left eq of bond eq: a = e.l.                                                                                                                                                                                                                                                                                                                                      |
| closedPivotal_exit_iff_connIn_on_shield              | Pivotal equivalence on a fixed shield value. On {Shield = S}, an oriented boundary bond (x,y) in boundaryE S is closed-pivotal for A_Lam iff 0 is connected to x inside S.                                                                                                                                                                                                                                                  |
| phiMinIn_le_phi                                      | The finite minimum is below each admissible phi p S.                                                                                                                                                                                                                                                                                                                                                                        |

| Name                                                   | Natural-language mathematical statement                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| prob_nonneg [private]                                  | Private supporting lemma proving nonnegativity for prob nonneg: 0 is at most Prob p E.                                                                                                                                                                                                                                                                                                                    |
| prob_le_of_pred_imp [private]                          | Private supporting lemma proving an inequality for prob le of pred imp: Prob p E is at most Prob p F.                                                                                                                                                                                                                                                                                                     |
| prob_union_le [private]                                | Private supporting lemma proving an inequality for prob union le: Prob p (E.union F) is at most Prob p E + Prob p F.                                                                                                                                                                                                                                                                                      |
| prob_finsetUnionEvent_le_sum [private]                 | Private supporting lemma proving an inequality for prob finset Union Event le sum: Prob p (finsetUnionEvent s E) is at most s.sum fun i => Prob p (E i).                                                                                                                                                                                                                                                  |
| sum_prob_le_prob_of_disjoint_subevents [private]       | Private supporting lemma proving an inequality for sum prob le prob of disjoint subevents: s.sum (fun i => Prob p (F i)) is at most Prob p E.                                                                                                                                                                                                                                                             |
| phi_nonneg [private]                                   | Private supporting lemma proving nonnegativity for phi nonneg: 0 is at most phi p S.                                                                                                                                                                                                                                                                                                                      |
| phiMinIn_nonneg [private]                              | Private supporting lemma proving nonnegativity for phi Min In nonneg: 0 is at most phiMinIn p Lam h0.                                                                                                                                                                                                                                                                                                     |
| subset_of_mem_finiteSubsetsWithZero [private]          | Private supporting lemma for subset of mem finite Subsets With Zero: S is at most Lam.                                                                                                                                                                                                                                                                                                                    |
| zero_mem_of_mem_finiteSubsetsWithZero [private]        | Private supporting lemma for zero mem of mem finite Subsets With Zero: (0 : Vertex d) belongs to S.                                                                                                                                                                                                                                                                                                       |
| prob_exit_compl_eq [private]                           | Private supporting lemma proving an equality for prob exit compl eq: Prob p ((exitEvent Lam h0).compl) = 1 - exitProb p Lam h0.                                                                                                                                                                                                                                                                           |
| one_sub_exitProb_le_sum_shield [private]               | Private supporting lemma proving an inequality for one sub exit Prob le sum shield: 1 - exitProb p Lam h0 is at most (finiteSubsetsWithZero Lam).attach.sum (fun S => Prob p (shieldEvent Lam S.1 (subset_of_mem_finiteSubsetsWithZero S.2))).                                                                                                                                                            |
| bondOfAdj_mem_exitSupport_of_boundary_subset [private] | Private supporting lemma for bond Of Adj mem exit Support of boundary subset: bondOfAdj (orientedBoundary_adj he) belongs to (exitEvent Lam (hS h0S)).support.                                                                                                                                                                                                                                            |
| orientedBoundary_bond_injective [private]              | Private supporting lemma for oriented Boundary bond injective: e = f.                                                                                                                                                                                                                                                                                                                                     |
| conn_shield_prob_eq_mul [private]                      | Private supporting lemma proving an equality for conn shield prob eq mul: Prob p ((connInEvent S (0 : Vertex d) e.1.1).inter (shieldEvent Lam S hS)) = Prob p (connInEvent S (0 : Vertex d) e.1.1) * Prob p (shieldEvent Lam S hS).                                                                                                                                                                       |
| conn_shield_le_closedPivotal_shield [private]          | Private supporting lemma proving an inequality for conn shield le closed Pivotal shield: Prob p ((connInEvent S (0 : Vertex d) e.1.1).inter (shieldEvent Lam S hS)) is at most Prob p (((closedPivotalEvent (exitEvent Lam h0) (bondOfAdj (orientedBoundary_adj e.2))).inter (shieldEvent Lam S hS))).                                                                                                    |
| shield_phiMin_div_mul_le_boundary_pivotal [private]    | Private supporting lemma proving an inequality for shield phi Min div mul le boundary pivotal: (phiMinIn p Lam h0 / p) * Prob p (shieldEvent Lam S hS) is at most (orientedBoundary S).attach.sum (fun e => Prob p (((closedPivotalEvent (exitEvent Lam h0) (bondOfAdj (orientedBoundary_adj e.2))).inter (shieldEvent Lam S hS)))).                                                                      |
| boundary_pivotal_shield_sum_le_support_sum [private]   | Private supporting lemma proving an inequality for boundary pivotal shield sum le support sum: (orientedBoundary S).attach.sum (fun e => Prob p (((closedPivotalEvent (exitEvent Lam h0) (bondOfAdj (orientedBoundary_adj e.2))).inter (shieldEvent Lam S hS))))) is at most (exitEvent Lam h0).support.sum (fun b => Prob p (((closedPivotalEvent (exitEvent Lam h0) b).inter (shieldEvent Lam S hS)))). |
| shield_phiMin_div_mul_le_support_pivotal [private]     | Private supporting lemma proving an inequality for shield phi Min div mul le support pivotal: (phiMinIn p Lam h0 / p) * Prob p (shieldEvent Lam S hS) is at most (exitEvent Lam h0).support.sum (fun b => Prob p (((closedPivotalEvent (exitEvent Lam h0) b).inter (shieldEvent Lam S hS)))).                                                                                                             |
| exit_russo_sum_lower_bound                             | The shield decomposition gives a lower bound on the Russo closed-pivotal sum for the finite exit event.                                                                                                                                                                                                                                                                                                   |
| differential_inequality                                | For a finite volume Lambda containing the origin, the derivative of the exit probability is bounded below by phiMinIn(p,Lambda)/(p(1-p)) times one minus the exit probability.                                                                                                                                                                                                                            |

Sharpness/OdeComparison.lean

Imports. Mathlib.

Definitions, structures, and abbreviations. *None.*

Theorems and lemmas.

| Name               | Natural-language mathematical statement                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ode_log_comparison | A real-valued function satisfying the differential lower bound $f'(t) \geq (1 - f(t))/(t(1 - t))$ obeys the corresponding logarithmic comparison inequality between $q$ and $p$ . |

Sharpness/Supercritical.lean

Imports. Sharpness.DiffIneq, Sharpness.OdeComparison.

Definitions, structures, and abbreviations. *None.*

Theorems and lemmas.

| Name                                         | Natural-language mathematical statement                                                                                                                                   |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bernWeight_all_false_pos [private]           | Private supporting lemma for bern Weight all false pos: $0 < \text{bernWeight } p \text{ (fun } \_ : \alpha \Rightarrow \text{false})$ .                                  |
| bernProb_pos_of_all_false [private]          | Private supporting lemma for bern Prob pos of all false: $0 < \text{bernProb } p \text{ } A$ .                                                                            |
| local_prob_lt_one_of_all_false_not [private] | Private supporting lemma proving a strict inequality for local prob lt one of all false not: $\text{Prob } p \text{ } E < 1$ .                                            |
| exitProb_lt_one_of_lt_one                    | Finite exit events have probability strictly below one when $p < 1$ : the all-closed assignment closes, in particular, every boundary edge.                               |
| one_le_phiMinIn_of_pTilde_lt [private]       | Private supporting lemma proving an inequality for one le phi Min In of p Tilde lt: $1$ is at most $\text{phiMinIn } t \text{ } \text{Lam } h0$ .                         |
| pTilde_endpoint_bound [private]              | Private supporting lemma for p Tilde endpoint bound: $(p - a) / (p * (1 - a))$ is at most $x$ .                                                                           |
| finite_exit_lower_bound_above_q [private]    | Private supporting lemma for finite exit lower bound above $q$ : $(p - q) / (p * (1 - q))$ is at most $\text{exitProb } p \text{ } \text{Lam } h0$ .                      |
| supercritical_lower_bound_above_pTilde       | Above the finite-set critical point pTilde and below $1$ , $\theta$ is bounded below by the explicit mean-field expression $(p - p\text{Tilde})/(p(1 - p\text{Tilde}))$ . |

Sharpness/BoundaryIneq.lean

Imports. Sharpness.Clusters, Sharpness.Events, Sharpness.Independence.

Definitions, structures, and abbreviations.

| Name                                      | Natural-language mathematical statement                |
|-------------------------------------------|--------------------------------------------------------|
| finsetUnionEvent [private, noncomputable] | Private auxiliary definition named finset Union Event. |
| clusterEvent [private, noncomputable]     | Private auxiliary definition named cluster Event.      |
| edgeOpenEvent [private, noncomputable]    | Private auxiliary definition named edge Open Event.    |

Theorems and lemmas.

| Name                                                 | Natural-language mathematical statement                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| prob_le_of_pred_imp [private]                        | Private supporting lemma proving an inequality for prob le of pred imp: Prob p E is at most Prob p F.                                                                                                                                                                                                                                                                                                                                                        |
| prob_union_le [private]                              | Private supporting lemma proving an inequality for prob union le: Prob p (E.union F) is at most Prob p E + Prob p F.                                                                                                                                                                                                                                                                                                                                         |
| prob_finsetUnionEvent_le_sum [private]               | Private supporting lemma proving an inequality for prob finset Union Event le sum: Prob p (finsetUnionEvent s E) is at most s.sum fun i => Prob p (E i).                                                                                                                                                                                                                                                                                                     |
| prob_nonneg [private]                                | Private supporting lemma proving nonnegativity for prob nonneg: 0 is at most Prob p E.                                                                                                                                                                                                                                                                                                                                                                       |
| prob_edgeOpenEvent_eq [private]                      | Private supporting lemma proving an equality for prob edge Open Event eq: Prob p (edgeOpenEvent b) = p.                                                                                                                                                                                                                                                                                                                                                      |
| mem_of_mem_internalBonds_carrier [private]           | Private supporting lemma for mem of mem internal Bonds carrier: x belongs to S.                                                                                                                                                                                                                                                                                                                                                                              |
| clusterIncidentBonds_subset_internalBonds [private]  | Private supporting lemma for cluster Incident Bonds subset internal Bonds: clusterIncidentBonds S C is at most internalBonds S.                                                                                                                                                                                                                                                                                                                              |
| exists_carrier_mem_of_clusterIncidentBonds [private] | Private supporting lemma proving existence of the stated object for exists carrier mem of cluster Incident Bonds: there exists x, x belongs to b.carrier and x belongs to C.                                                                                                                                                                                                                                                                                 |
| disjoint_clusterIncident_boundaryBond [private]      | Private supporting lemma for disjoint cluster Incident boundary Bond: Disjoint (clusterIncidentBonds S C) ({bondOfAdj (orientedBoundary_adj he)} : Finset (Bond d)).                                                                                                                                                                                                                                                                                         |
| disjoint_clusterIncident_internal_sdiff [private]    | Private supporting lemma for disjoint cluster Incident internal sdiff: Disjoint (clusterIncidentBonds S C) (internalBonds (T \ C)).                                                                                                                                                                                                                                                                                                                          |
| disjoint_boundaryBond_internal_sdiff [private]       | Private supporting lemma for disjoint boundary Bond internal sdiff: Disjoint ({bondOfAdj (orientedBoundary_adj he)} : Finset (Bond d)) (internalBonds (T \ C)).                                                                                                                                                                                                                                                                                              |
| target_mem_of_connIn [private]                       | Private supporting lemma for target mem of conn In: x belongs to S.                                                                                                                                                                                                                                                                                                                                                                                          |
| clusterIn_subset [private]                           | Private supporting lemma for cluster In subset: clusterIn omega S u is at most S.                                                                                                                                                                                                                                                                                                                                                                            |
| connIn_subset [private]                              | Private supporting lemma for conn In subset: ConnIn omega T x y.                                                                                                                                                                                                                                                                                                                                                                                             |
| connToSetIn_subset [private]                         | Private supporting lemma for conn To Set In subset: ConnToSetIn omega T u B.                                                                                                                                                                                                                                                                                                                                                                                 |
| connToSetIn_sdiff_subset [private]                   | Private supporting lemma for conn To Set In sdiff subset: ConnToSetIn omega T u B.                                                                                                                                                                                                                                                                                                                                                                           |
| connIn_single [private]                              | Private supporting lemma for conn In single: ConnIn omega S x x.                                                                                                                                                                                                                                                                                                                                                                                             |
| cluster_subset [private]                             | Private supporting lemma for cluster subset: x belongs to S.                                                                                                                                                                                                                                                                                                                                                                                                 |
| conn_of_mem_cluster [private]                        | Private supporting lemma for conn of mem cluster: ConnIn omega S u x.                                                                                                                                                                                                                                                                                                                                                                                        |
| mem_cluster_of_conn [private]                        | Private supporting lemma for mem cluster of conn: x belongs to C.                                                                                                                                                                                                                                                                                                                                                                                            |
| sum_clusterEvent_le_connIn [private]                 | Private supporting lemma proving an inequality for sum cluster Event le conn In: ((S.powerset.filter fun C => x belongs to C).sum (fun C => Prob p (clusterEvent S C u))) is at most Prob p (connInEvent S u x).                                                                                                                                                                                                                                             |
| connIn_append_open_adj_same [private]                | Private supporting lemma for conn In append open adj same: ConnIn omega S x z.                                                                                                                                                                                                                                                                                                                                                                               |
| cluster_closed_under_open_adj [private]              | Private supporting lemma for cluster closed under open adj: y belongs to C.                                                                                                                                                                                                                                                                                                                                                                                  |
| path_last_exit_cluster [private]                     | Private supporting lemma for path last exit cluster: for every {gamma : List (Vertex d)} {x b : Vertex d}, PathFromTo gamma x b implies PathIn T gamma implies OpenPath omega gamma implies (there exists z, z belongs to gamma and z belongs to C) implies b does not belong to C implies there exists a y : Vertex d, there exists hxy : Adj a y, a belongs to C and y does not belong to C and omega (bondOfAdj hxy) = true and ConnIn omega (T \ C) y b. |
| connToSet_cluster_boundary_witness [private]         | Private supporting lemma for conn To Set cluster boundary witness: there exists e, there exists he : e belongs to orientedBoundary S, e.1 belongs to C and omega (bondOfAdj (orientedBoundary_adj he)) = true and ConnToSetIn omega (T \ C) e.2 B.                                                                                                                                                                                                           |

| Name                                            | Natural-language mathematical statement                                                                                                                                                                                                                                                          |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| boundary_triple_prob_le<br>[private]            | Private supporting lemma proving an inequality for boundary triple prob le: Prob p (((clusterEvent S C u).inter (edgeOpenEvent (bondOfAdj (orientedBoundary _adj he))))).inter (connToSetInEvent (T \ C) e.2 B)) is at most Prob p (clusterEvent S C u) * p * Prob p (connToSetInEvent T e.2 B). |
| boundary_cluster_value_bound<br>[private]       | Private supporting lemma for boundary cluster value bound: Prob p ((clusterEvent S C u).inter (connToSetInEvent T u B)) is at most (orientedBoundary S).sum (fun e => if _ : e.1 belongs to C then Prob p (clusterEvent S C u) * p * Prob p (connToSetInEvent T e.2 B) else 0).                  |
| boundary_inequality_cluster_expansion [private] | Cluster last-exit expansion for the finite-volume boundary inequality.                                                                                                                                                                                                                           |
| boundary_inequality                             | The probability of connecting from u to B inside T is bounded by a sum over the oriented boundary of S, with one factor for reaching the boundary point in S, one factor p for the boundary bond, and one factor for connecting onward inside T.                                                 |

## Sharpness/Subcritical.lean

Imports. Sharpness.BoundaryIneq, Sharpness.FiniteGeometry, Sharpness.Phi.

## Definitions, structures, and abbreviations.

| Name                                       | Natural-language mathematical statement                            |
|--------------------------------------------|--------------------------------------------------------------------|
| expScale [private,<br>noncomputable]       | Private auxiliary definition named exp Scale.                      |
| translateVertexEmbedding<br>[private]      | Private auxiliary definition named translate Vertex Embedding.     |
| translateBond [private]                    | Private auxiliary definition named translate Bond.                 |
| translateInternalBondsEquiv<br>[private]   | Private auxiliary definition named translate Internal Bonds Equiv. |
| boxExitTargets [private,<br>noncomputable] | Private auxiliary definition named box Exit Targets.               |
| boxExitAmbient [private,<br>noncomputable] | Private auxiliary definition named box Exit Ambient.               |

## Theorems and lemmas.

| Name                                         | Natural-language mathematical statement                                                                                          |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| bernWeight_zero_of_all_false<br>[private]    | Private supporting lemma for bern Weight zero of all false: bernWeight (0 : Real) sigma = 1.                                     |
| bernWeight_zero_of_exists_true<br>[private]  | Private supporting lemma for bern Weight zero of exists true: bernWeight (0 : Real) sigma = 0.                                   |
| exists_true_of_ne_all_false<br>[private]     | Private supporting lemma proving existence of the stated object for exists true of ne all false: there exists e, sigma e = true. |
| local_prob_zero_of_all_false_not [private]   | Private supporting lemma for local prob zero of all false not: Prob (0 : Real) E = 0.                                            |
| boxExitProb_zero [private]                   | Private supporting lemma for box Exit Prob zero: boxExitProb d (0 : Real) n = 0.                                                 |
| boxExitProb_le_one [private]                 | Private supporting lemma proving an upper bound by one for box Exit Prob le one: boxExitProb d p n is at most 1.                 |
| bernWeight_all_false_pos<br>[private]        | Private supporting lemma for bern Weight all false pos: 0 < bernWeight p (fun _ : alpha => false).                               |
| bernProb_pos_of_all_false<br>[private]       | Private supporting lemma for bern Prob pos of all false: 0 < bernProb p A.                                                       |
| local_prob_lt_one_of_all_false_not [private] | Private supporting lemma proving a strict inequality for local prob lt one of all false not: Prob p E < 1.                       |
| boxExitProb_lt_one_of_lt_one<br>[private]    | Private supporting lemma proving a strict inequality for box Exit Prob lt one of lt one: boxExitProb d p n < 1.                  |



| Name                                       | Natural-language mathematical statement                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| div_step_eq [private]                      | Private supporting lemma proving an equality for div step eq: $(n - L) / L + 1 = n / L$ .                                                                                                                                                                                                                                                                                                                                                                     |
| recurrence_iterate [private]               | Private supporting lemma for recurrence iterate: for every $n : \text{Nat}$ , a $n$ is at most $\rho \wedge (n / L)$ .                                                                                                                                                                                                                                                                                                                                        |
| exp_bound_single [private]                 | Private supporting lemma for exp bound single: a is at most $\text{Real.exp } (-(c * (n : \text{Real})))$ .                                                                                                                                                                                                                                                                                                                                                   |
| exists_small_exp_bound [private]           | Private supporting lemma proving existence of the stated object for exists small exp bound: there exists $c : \text{Real}$ , $0 < c$ and for every $n : \text{Nat}$ , $n < L$ implies a $n$ is at most $\text{Real.exp } (-(c * (n : \text{Real})))$ .                                                                                                                                                                                                        |
| rho_pow_le_exp [private]                   | Private supporting lemma proving an inequality for rho pow le exp: $\rho \wedge (n / L)$ is at most $\text{Real.exp } (-(c * (n : \text{Real})))$ .                                                                                                                                                                                                                                                                                                           |
| recurrence_exponential_bound [private]     | Private supporting lemma for recurrence exponential bound: there exists $c : \text{Real}$ , $0 < c$ and for every $n : \text{Nat}$ , a $n$ is at most $\text{Real.exp } (-(c * (n : \text{Real})))$ .                                                                                                                                                                                                                                                         |
| prob_le_of_pred_imp [private]              | Private supporting lemma proving an inequality for prob le of pred imp: $\text{Prob } p$ E is at most $\text{Prob } p$ F.                                                                                                                                                                                                                                                                                                                                     |
| translateVertex_neg_apply [private]        | Private supporting lemma for translate Vertex neg apply: $\text{translateVertex } (-a)$ $(\text{translateVertex } a \ x) = x$ .                                                                                                                                                                                                                                                                                                                               |
| translateVertex_neg_self [private]         | Private supporting lemma for translate Vertex neg self: $\text{translateVertex } (-x) \ x = (0 : \text{Vertex } d)$ .                                                                                                                                                                                                                                                                                                                                         |
| translateVertex_neg_eq_sub [private]       | Private supporting lemma proving an equality for translate Vertex neg eq sub: $\text{translateVertex } (-y) \ x = x - y$ .                                                                                                                                                                                                                                                                                                                                    |
| translateVertex_neg_image [private]        | Private supporting lemma for translate Vertex neg image: $(S.\text{image } (\text{translateVertex } a)).\text{image } (\text{translateVertex } (-a)) = S$ .                                                                                                                                                                                                                                                                                                   |
| translateBond_neg_apply [private]          | Private supporting lemma for translate Bond neg apply: $\text{translateBond } (-a)$ $(\text{translateBond } a \ b) = b$ .                                                                                                                                                                                                                                                                                                                                     |
| translateBond_bondOfAdj [private]          | Private supporting lemma for translate Bond bond Of Adj: $\text{translateBond } a$ $(\text{bondOfAdj } hxy) = \text{bondOfAdj } ((\text{adj\_translate\_iff } a)$ .                                                                                                                                                                                                                                                                                           |
| translateBond_mem_ [private]               | Private supporting lemma for translate Bond mem internal Bonds: $\text{translateBond } a \ b$ belongs to $\text{internalBonds } (S.\text{image } (\text{translateVertex } a))$ .                                                                                                                                                                                                                                                                              |
| internalBonds [private]                    | Private supporting lemma for translate Bond mem internal Bonds neg: $\text{translateBond } (-a) \ b$ belongs to $\text{internalBonds } S$ .                                                                                                                                                                                                                                                                                                                   |
| translateBond_mem_ [private]               | Private supporting lemma for open Path translate config: $\{d : \text{Nat}\} \ \{\omega \ \omega' : \text{Config } d\} \ \{a : \text{Vertex } d\} \ \{\gamma : \text{List } (\text{Vertex } d)\} \ (\text{hopen\_translate} : \text{for every } \{u \ v : \text{Vertex } d\} \ (\text{huv} : \text{Adj } u \ v), \ \omega \ (\text{bondOfAdj } \text{huv}) = \text{true} \text{ implies } \omega' \ (\text{bondOfAdj } ((\text{adj\_translate\_iff } a)$ .    |
| openPath_translate_config [private]        | Private supporting lemma for conn In translate config: $\{d : \text{Nat}\} \ \{\omega \ \omega' : \text{Config } d\} \ \{S : \text{Finset } (\text{Vertex } d)\} \ \{a \ x \ y : \text{Vertex } d\} \ (\text{hopen\_translate} : \text{for every } \{u \ v : \text{Vertex } d\} \ (\text{huv} : \text{Adj } u \ v), \ \omega \ (\text{bondOfAdj } \text{huv}) = \text{true} \text{ implies } \omega' \ (\text{bondOfAdj } ((\text{adj\_translate\_iff } a)$ . |
| connIn_translate_config [private]          | Private supporting lemma for conn To Set In translate assignment: let $\sigma : \text{internalBonds } T$ implies $\text{Bool}$ .                                                                                                                                                                                                                                                                                                                              |
| connToSetIn_translate_assignment [private] | Private supporting lemma proving an inequality for prob conn To Set In le translate: $\text{Prob } p$ $(\text{connToSetInEvent } T \ u \ B)$ is at most $\text{Prob } p$ $(\text{connToSetInEvent } (T.\text{image } (\text{translateVertex } a)) \ (\text{translateVertex } a \ u) \ (B.\text{image } (\text{translateVertex } a)))$ .                                                                                                                       |
| prob_connToSetIn_le_translate [private]    | Private supporting lemma for conn In single: $\text{ConnIn } \omega \ S \ x \ x$ .                                                                                                                                                                                                                                                                                                                                                                            |
| connIn_single [private]                    | Private supporting lemma for conn In prepend open adj: $\text{ConnIn } \omega \ S \ x \ z$ .                                                                                                                                                                                                                                                                                                                                                                  |
| connIn_prepend_open_adj [private]          | Private supporting lemma for conn In append open adj: $\text{ConnIn } \omega \ S \ x \ z$ .                                                                                                                                                                                                                                                                                                                                                                   |
| connIn_append_open_adj [private]           | Private supporting lemma for exit Pred of conn In to not mem: there exists $e$ , there exists $he : e$ belongs to $\text{orientedBoundary } S$ , $\text{ConnIn } \omega \ S \ x \ e.1$ and $\omega \ (\text{bondOfAdj } (\text{orientedBoundary\_adj } he)) = \text{true}$ .                                                                                                                                                                                  |
| exitPred_of_connIn_to_not_mem [private]    | Private supporting lemma for mem box Exit Targets not ball: $x$ does not belong to ball $d \ n$ .                                                                                                                                                                                                                                                                                                                                                             |
| mem_boxExitTargets_not_ball [private]      |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

| Name                                                    | Natural-language mathematical statement                                                                                                                                                                     |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| boxExitProb_le_boundary_conn [private]                  | Private supporting lemma proving an inequality for box Exit Prob le boundary conn: boxExitProb d p n is at most Prob p (connToSetInEvent (boxExitAmbient d n) (0 : Vertex d) (boxExitTargets d n)).         |
| connToSetIn_le_boxExitProb_of_targets_outside [private] | Private supporting lemma proving an inequality for conn To Set In le box Exit Prob of targets outside: Prob p (connToSetInEvent T (0 : Vertex d) B) is at most boxExitProb d p k.                           |
| connToBoxBoundary_le_shifted_boxExitProb [private]      | Private supporting lemma proving an inequality for conn To Box Boundary le shifted box Exit Prob: Prob p (connToSetInEvent (boxExitAmbient d n) y (boxExitTargets d n)) is at most boxExitProb d p (n - L). |
| boxExitProb_recurrence_of_phi_lt_one [private]          | Private supporting lemma proving a strict inequality for box Exit Prob recurrence of phi lt one: for every n : Nat, L is at most n implies boxExitProb d p n is at most phi p S * boxExitProb d p (n - L).  |
| exists_ball_bound_for_finset [private]                  | Private supporting lemma proving existence of the stated object for exists ball bound for finset: there exists L : Nat, 0 < L and for every x, x belongs to S implies x belongs to ball d (L - 1).          |
| exponential_decay_below_pTilde_core [private]           | Private supporting lemma for exponential decay below p Tilde core: there exists c : Real, 0 < c and for every n : Nat, boxExitProb d p n is at most Real.exp (-(c * (n : Real))).                           |
| exponential_decay_below_pTilde                          | Below the finite-set critical point pTilde, finite box exit probabilities decay exponentially.                                                                                                              |

Sharpness/CriticalPoint.lean

Imports. Sharpness.Subcritical, Sharpness.Supercritical.

#### Definitions, structures, and abbreviations.

| Name                  | Natural-language mathematical statement                                                 |
|-----------------------|-----------------------------------------------------------------------------------------|
| pCrit [noncomputable] | The theta-critical point, defined as the infimum of parameters where theta is positive. |
| axisVertex [private]  | Private auxiliary definition named axis Vertex.                                         |

#### Theorems and lemmas.

| Name                                     | Natural-language mathematical statement                                                                                           |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| pCritSet_bddBelow [private]              | Private supporting lemma for p Crit Set bdd Below: BddBelow {p : Real   0 is at most p and p is at most 1 and 0 < theta d p}.     |
| bernWeight_one_of_all_true [private]     | Private supporting lemma for bern Weight one of all true: bernWeight (1 : Real) sigma = 1.                                        |
| bernWeight_one_of_exists_false [private] | Private supporting lemma for bern Weight one of exists false: bernWeight (1 : Real) sigma = 0.                                    |
| exists_false_of_ne_all_true [private]    | Private supporting lemma proving existence of the stated object for exists false of ne all true: there exists e, sigma e = false. |
| local_prob_one_of_all_true [private]     | Private supporting lemma for local prob one of all true: Prob (1 : Real) E = 1.                                                   |
| axisVertex_zero [private]                | Private supporting lemma for axis Vertex zero: axisVertex i 0 = (0 : Vertex d).                                                   |
| axisVertex_l1 [private]                  | Private supporting lemma for axis Vertex l1: l1 (axisVertex i k) = k.                                                             |
| axisVertex_adj_succ [private]            | Private supporting lemma for axis Vertex adj succ: Adj (axisVertex i k) (axisVertex i (k + 1)).                                   |
| List_range_succ_head [private]           | Private supporting lemma for List range succ head: (List.range (n + 1)).head? = some 0.                                           |
| axisPath_isPath [private]                | Private supporting lemma for axis Path is Path: IsPath ((List.range (n + 1)).map (axisVertex i)).                                 |
| axisPath_head [private]                  | Private supporting lemma for axis Path head: ((List.range (n + 1)).map (axisVertex i)).head? = some (0 : Vertex d).               |

| Name                                       | Natural-language mathematical statement                                                                                                                                                                                                                                                                              |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| axisPath_last [private]                    | Private supporting lemma for axis Path last: $((\text{List.range } (n + 1)).\text{map } (\text{axisVertex } i)).\text{getLast?} = \text{some } (\text{axisVertex } i \ n)$ .                                                                                                                                         |
| axisPath_pathFromTo [private]              | Private supporting lemma for axis Path path From To: $\text{PathFromTo } ((\text{List.range } (n + 1)).\text{map } (\text{axisVertex } i)) \ (0 : \text{Vertex } d) \ (\text{axisVertex } i \ n)$ .                                                                                                                  |
| axisPath_pathIn [private]                  | Private supporting lemma for axis Path path In: $\text{PathIn } (\text{ball } d \ n) \ ((\text{List.range } (n + 1)).\text{map } (\text{axisVertex } i))$ .                                                                                                                                                          |
| axisPath_open_extend_exitSupport [private] | Private supporting lemma for axis Path open extend exit Support: $\text{OpenPath } (\text{extendConfig } (\text{exitEvent } (\text{ball } d \ n) \ (\text{zero\_mem\_ball } d \ n)).\text{support } (\text{fun } \_ \Rightarrow \text{true})) \ ((\text{List.range } (n + 1)).\text{map } (\text{axisVertex } i))$ . |
| exitPred_all_open_support [private]        | Private supporting lemma for exit Pred all open support: $\text{ExitPred } (\text{ball } d \ n) \ (\text{extendConfig } (\text{exitEvent } (\text{ball } d \ n) \ (\text{zero\_mem\_ball } d \ n)).\text{support } (\text{fun } \_ \Rightarrow \text{true}))$ .                                                      |
| boxExitProb_one_of_pos_dim [private]       | Private supporting lemma for box Exit Prob one of pos dim: $\text{boxExitProb } d \ (1 : \text{Real}) \ n = 1$ .                                                                                                                                                                                                     |
| pCritSet_nonempty_of_pos_dim [private]     | Private supporting lemma for p Crit Set nonempty of pos dim: $(\{p : \text{Real} \mid 0 \text{ is at most } p \text{ and } p \text{ is at most } 1 \text{ and } 0 < \text{theta } d \ p\} : \text{Set Real}).\text{Nonempty}$ .                                                                                      |
| pCrit_le_one [private]                     | Private supporting lemma proving an upper bound by one for p Crit le one: $p\text{Crit } d \text{ is at most } 1$ .                                                                                                                                                                                                  |
| pCrit_le_of_mem [private]                  | Private supporting lemma proving an inequality for p Crit le of mem: $p\text{Crit } d \text{ is at most } p$ .                                                                                                                                                                                                       |
| pCrit_le_pTilde [private]                  | Private supporting lemma proving an inequality for p Crit le p Tilde: $p\text{Crit } d \text{ is at most } p\text{Tilde } d$ .                                                                                                                                                                                       |
| pTilde_le_pCrit [private]                  | Private supporting lemma proving an inequality for p Tilde le p Crit: $p\text{Tilde } d \text{ is at most } p\text{Crit } d$ .                                                                                                                                                                                       |
| pTilde_eq_pCrit                            | In positive dimension, the finite-set critical point equals the theta-critical point.                                                                                                                                                                                                                                |
| exponential_decay_below_pCrit              | If $d > 0$ and $0 \leq p < p\_c(d)$ , then there is a positive constant $c$ such that $\text{boxExitProb}(d, p, n) \leq \exp(-c \ n)$ for every $n$ .                                                                                                                                                                |
| supercritical_lower_bound_above_pCrit      | If $d > 0$ and $p\_c(d) < p < 1$ , then $\text{theta\_d}(p) \geq (p - p\_c(d))/(p(1 - p\_c(d)))$ .                                                                                                                                                                                                                   |

## Sharpness/Main.lean

Imports. Sharpness.CriticalPoint.

Definitions, structures, and abbreviations. *None.*

## Theorems and lemmas.

| Name         | Natural-language mathematical statement                                                                                                                                                                                      |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sharpness_zd | For every dimension $d \geq 2$ , subcritical finite box exit probabilities decay exponentially below $p\_c(d)$ , and above $p\_c(d)$ the density $\text{theta\_d}(p)$ is bounded below by $(p - p\_c(d))/(p(1 - p\_c(d)))$ . |